

Homework 1 Summary

Logan Clevenger

CPSC – 224 – 02

9/16/2026

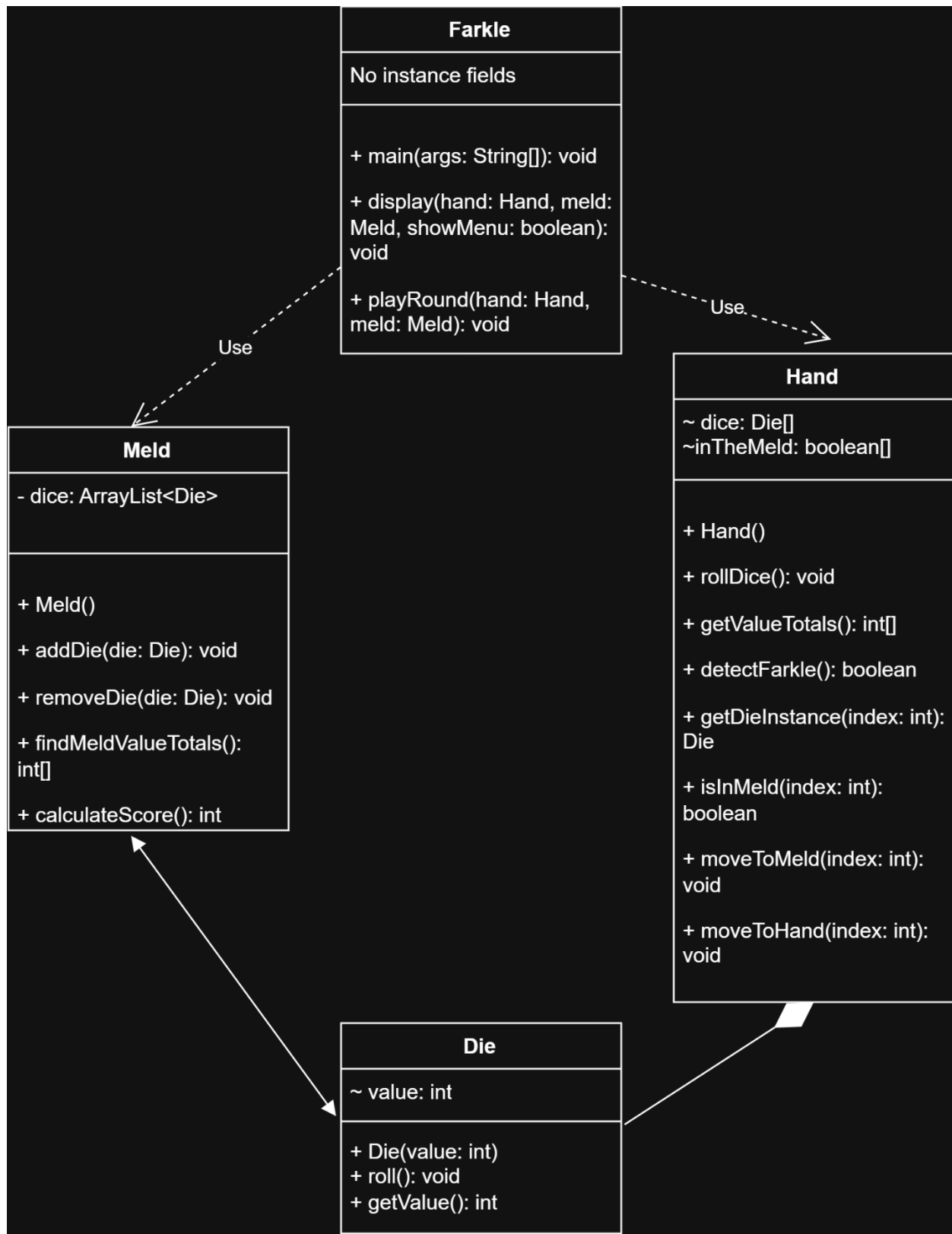
Goal/Purpose of Program:

The goal for this program is for a single player to be able to play one round of the game *Farkle*. The player should be able to roll 6 dice and see them displayed in their hand. From there, the player should be able to choose to switch dice between their hand and the meld. Once the player is satisfied with the dice in their meld, they should be able to bank their meld and see how many points they scored (using the Zag Farkle scoring rules).

Overview of General Program Design:

For this project, I utilized 4 different classes. I first made a Die class that represents one die in the *Farkle* game. This class stores its current numeric value and is able to roll randomly to another value (between 1-6). I then made a Hand class that owns 6 die instances. This class also tracks selections being made between the player's hand and their meld along with whether or not the player rolled a Farkle. Following that, I made a Meld class. This class stores selected dice from the player's hand and calculates the Zag Farkle score of the dice currently contained in the player's meld. Finally, I finished with the Farkle class. This class actually runs the game and controls menu display and user input for die selections.

UML Class Diagram (source: app.diagrams.net):



Major Design/Programming Challenges and Solutions:

1: One of my first major challenges was understanding how I could keep track of the dice that were currently in the player's hand vs. the player's meld. I decided to solve this issue by having the Hand class keep track of whether the dice were currently inside the hand using a Boolean array. From this, I was able to then use an ArrayList in the Meld class, as this made it super easy to add/take away die instances.

2: I also struggled with scoring combinations. I was stuck on the idea of double counting dice within meld when finding a meld's score. But I eventually settled on checking special combos that used all of the dice available (like straights and triple doubles). From there I was able to check for 3+ value occurrences and then leftover 1 and 5 values.

Retrospective:

One of the things I would change looking at my project in retrospect was how I handled user input between A,B,C,D,E,F in the game menu. The logic I used to implement my solution here was pretty repetitive and used a lot of if and if else statements. In the future, if I got another chance at this project, I would look into writing a loop to perform this functionality so my code is less bulky.